UI BUILDER

AUTOMATIC CODE GENERATION TOOL TO DESIGN AND DEVELOP UI

Project ID: 2022-070



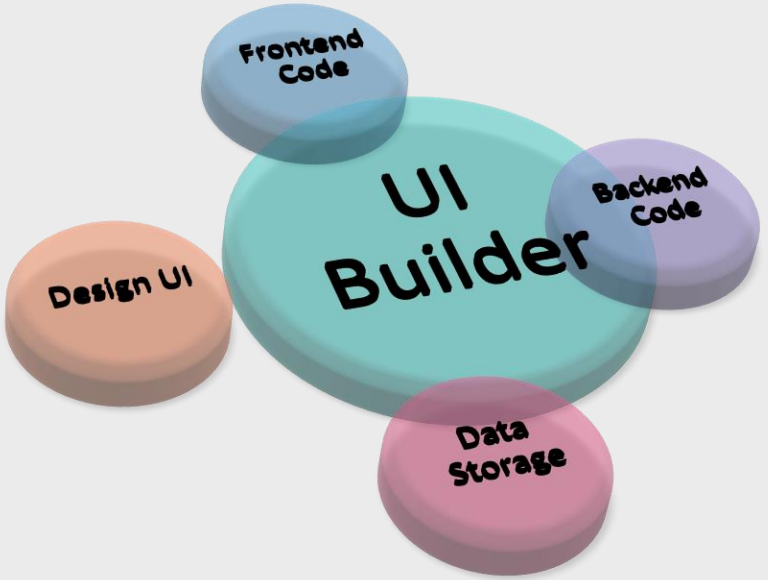
SLIIT
FACULTY OF COMPUTING

TEAM MEMBERS

Group ID: 2022-070	
Supervisor: Dr. Nuwan Kodagoda	
Co Supervisor: Ms. Kushnara Suriyawansa	
Student Name	**Student ID**
Ekanayake K.L.W (Group Leader)	IT19150758
Sakthivel.S	IT19158228
H.M. Kasuni Navodya	IT19144986
D.P Kavindi Gimshani	IT19150826

OUR RESEARCH

- Get insights to Automatic Code generation (ACG) concepts?
- Critical analysis of existing ACG tools and their usage in industry.
- Introduce a new ACG tool with enhanced features.
- Promote the introduced tool and deploy on production environment.



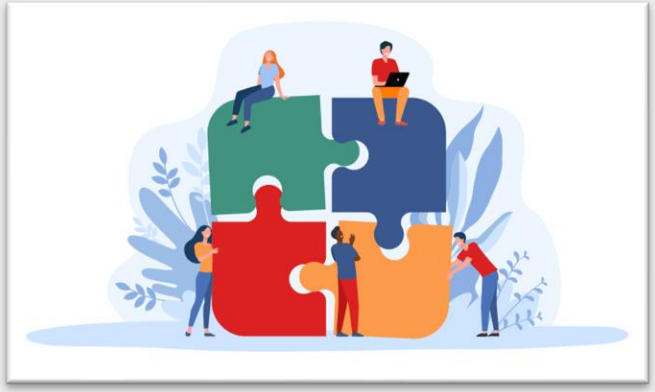
RESEARCH PROBLEM



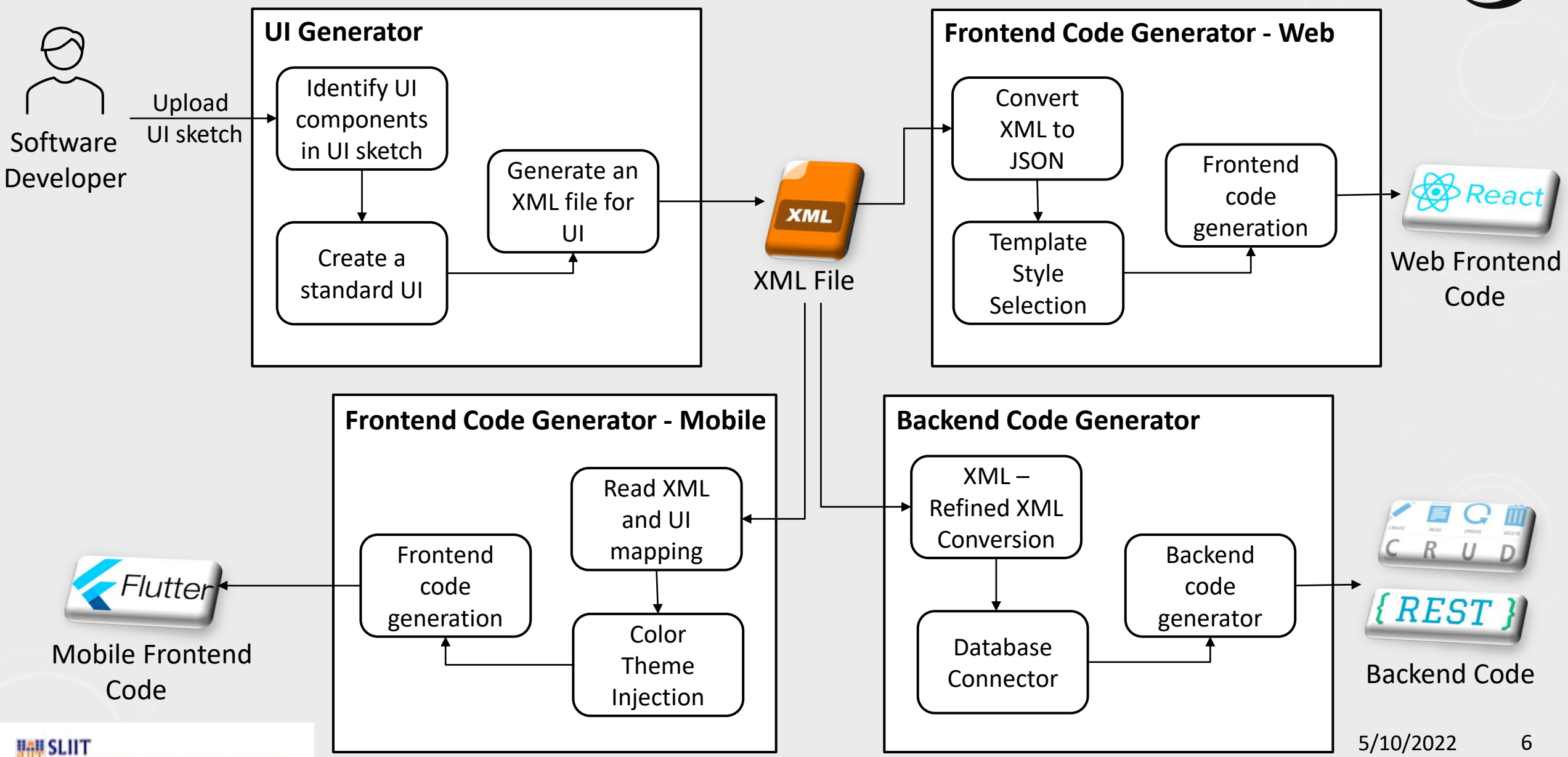
1. Need to follow the traditional method of software engineering process.
2. Available tools don’t fulfill the real requirements of software developers.
3. Auto-generated Code do not have the texture of real-handwritten code.
4. Auto-generated code only focuses on frontend, whereas backend code still needs to be written manually.

RESEARCH OBJECTIVES

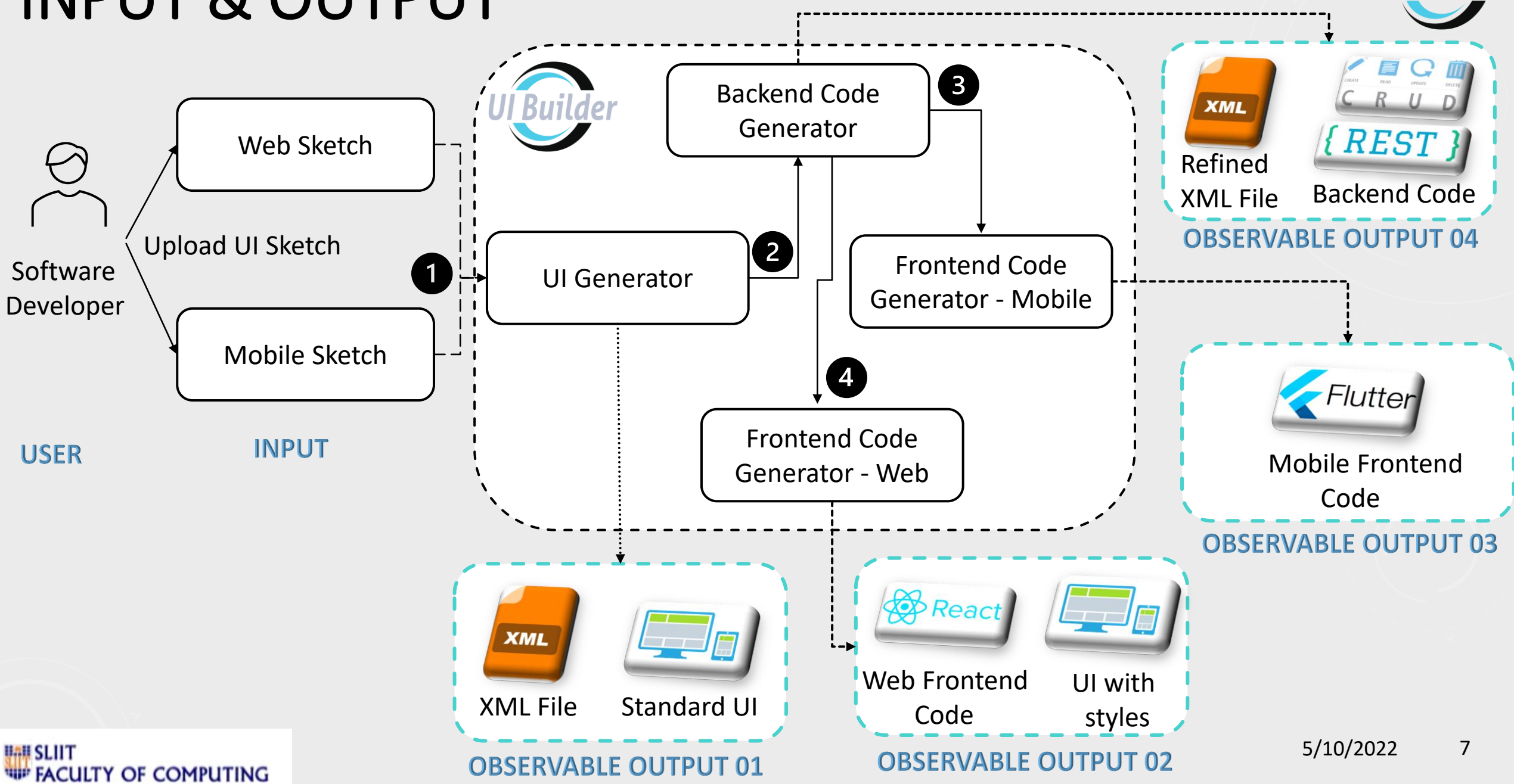
- Generate UI templates according to UI rough sketches given by users.
- Generate web frontend code from UI template using MERN Stack technologies.
- Generate mobile frontend code from UI template using code mapping.
- Develop backend code and DB generator adhering to software engineering principles.



SYSTEM OVERVIEW DIAGRAM



INPUT & OUTPUT



SOFTWARE DEVELOPMENT LIFECYCLE





Requirement Gathering

- Surveys and questionnaires.
- Interviews.
- Features from existing tools.

Design

- Sketches
- Wireframes
- Prototypes









Development

- Coding standards
- Code comments
- Code reviews



Testing

- Unit testing
- Integration testing
- System testing
- User acceptance testing



Maintenance

- User feedback
- Continuous improvements

OUR PLAN TO PROMOTE THE TOOL



- Conducting a live demonstration of code generation process in developer environment.
- Understanding the need and expectations of developers through automated feedback services integrated with the tool.
- Select startup companies and describe them the capabilities and advantages of using UI builder tool for their software development process.
- Use social media and publish an official website for the tool to make it reach out for wide range of people.
- Conduct a workshop on the quick setup of the tool in their PC for 1st and 2nd year IT students.
- Speak around '**Anyone Can Code**' concept to make It stand unique among the other code generation tools.
- Use quick startup guides and YouTube tutorials for better usability and user-friendliness.



IT19150758|EKANAYAKE K.L.W

BSC (HONS) DEGREE IN INFORMATION TECHNOLOGY

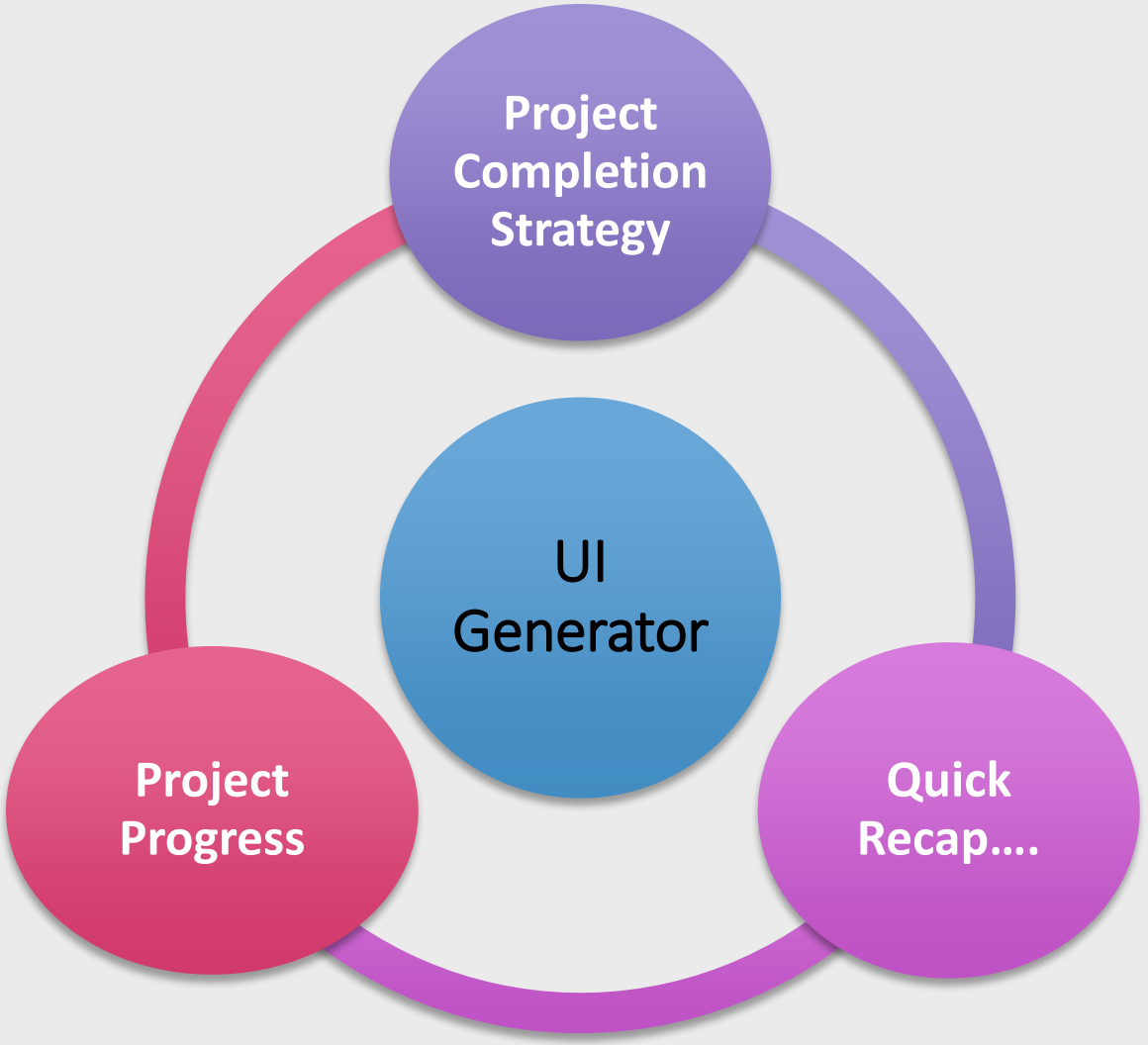
(SPECIALIZATION IN SOFTWARE ENGINEERING)

RESEARCH COMPONENT

GENERATING STANDARD UIs ACCORDING TO ROUGH SKETCHES.



INTRODUCTION



QUICK RECAP.....

Research



- Introduce a standard UI template generator according to the rough Uis.

Research Question



- How to generate standard UI according to rough UIs.

Objectives



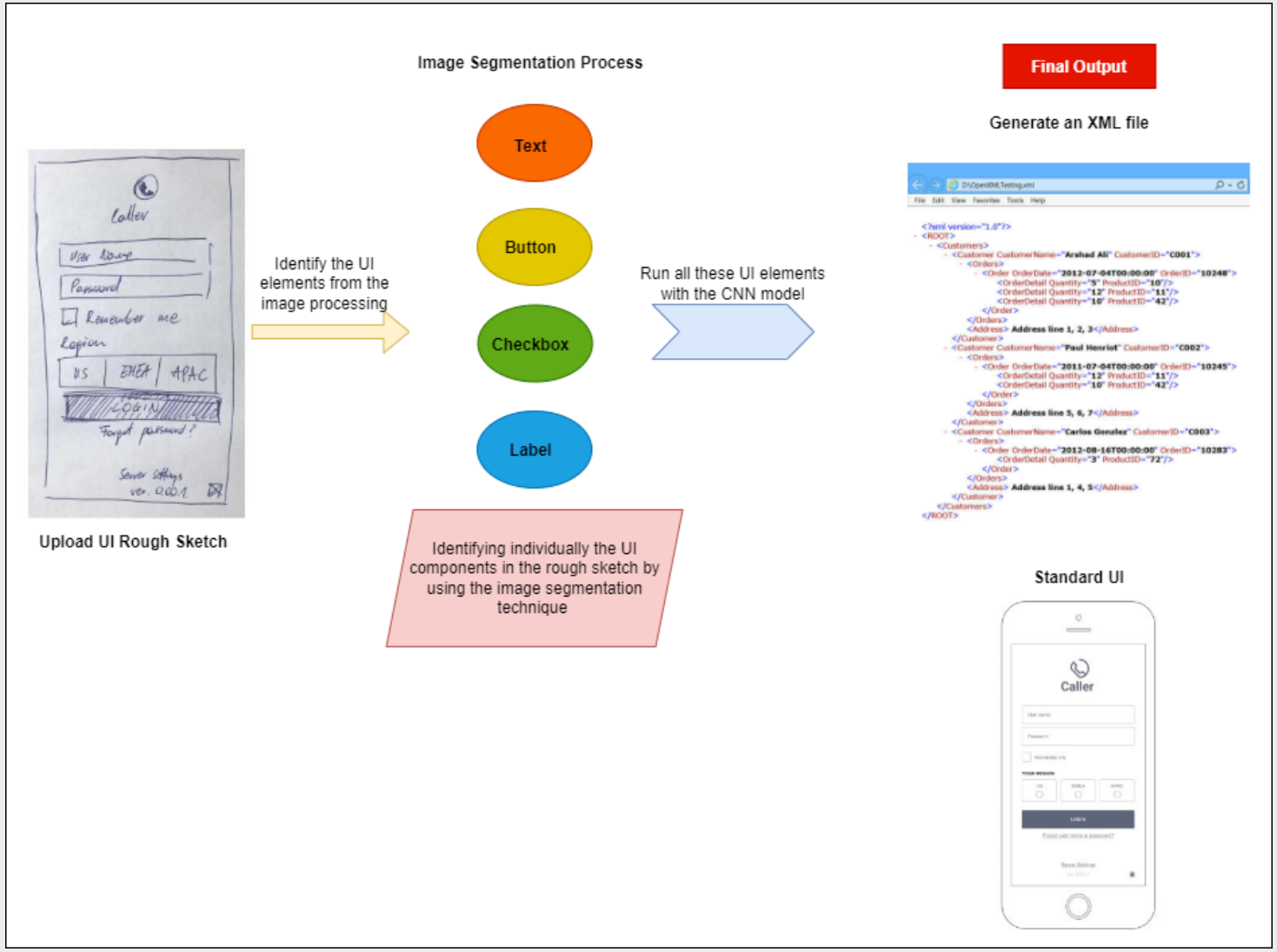
- Generate a standard UI according to rough UIs.

Research Gap

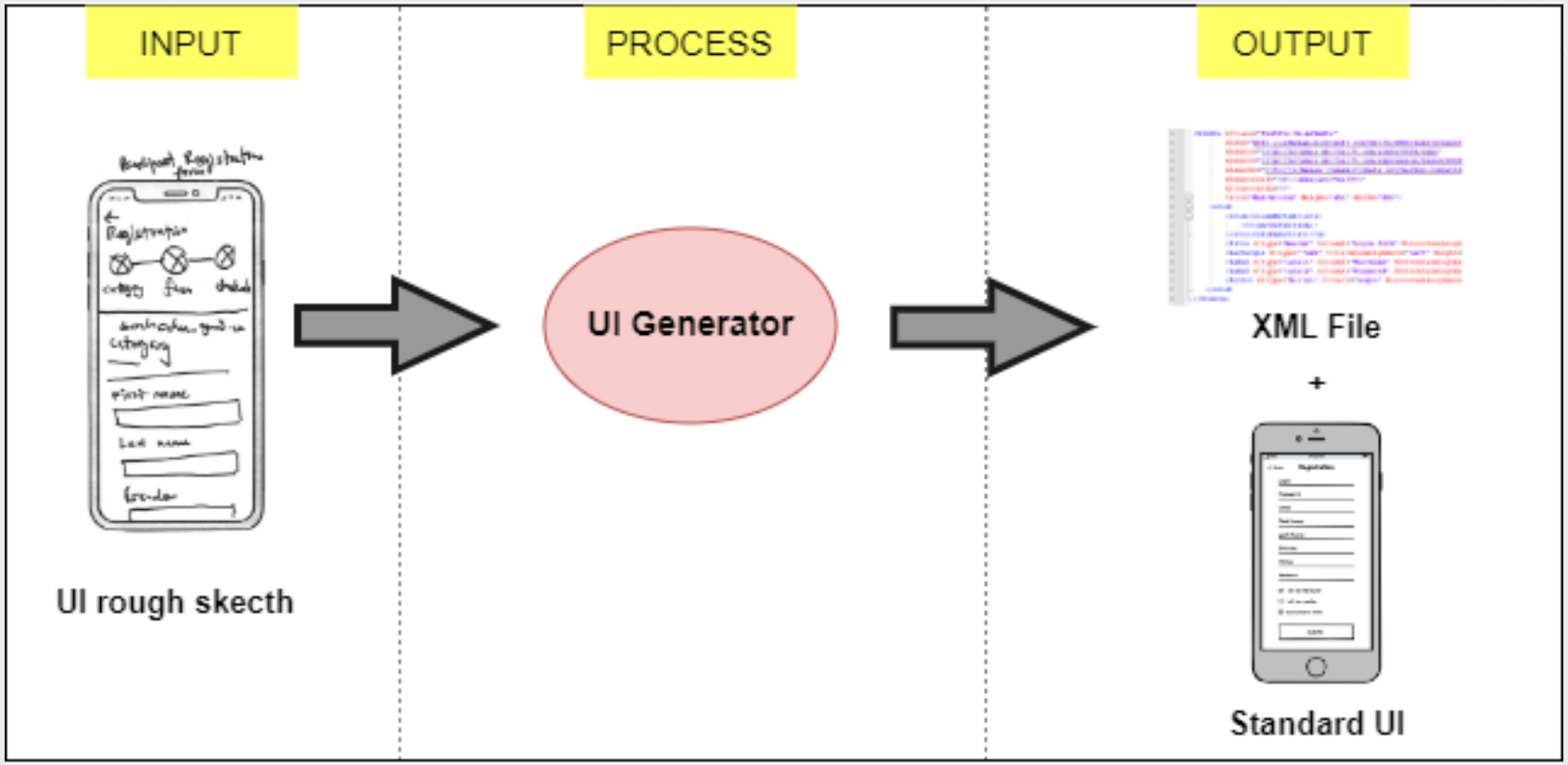


- Existing UI generator tools unable to convert rough UIs to standard Uis.

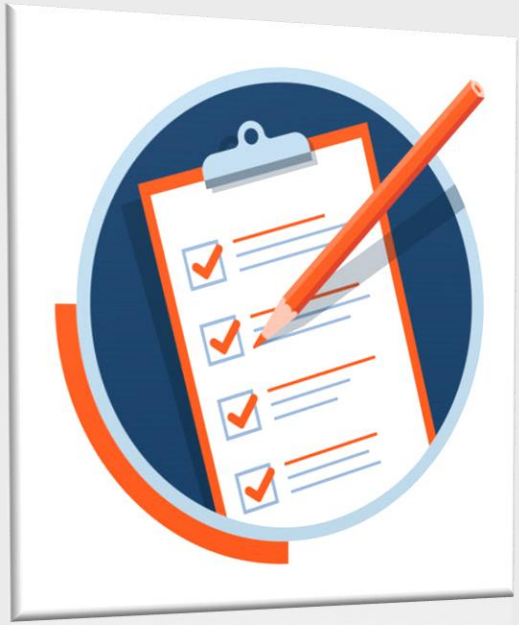
PROCESS


Image Segmentation Process
Final Output
Generate an XML file
Text
Button
Checkbox
Label
Identify the UI elements from the image processing
Run all these UI elements with the CNN model
Upload UI Rough Sketch
Identifying individually the UI components in the rough sketch by using the image segmentation technique
Standard UI
Caller

INPUT OUTPUT



REQUIREMENTS



- Identify the UI elements in the rough sketch using the CNN model.
- Get the accuracy values of each UI component.
- Map the UI elements to standard UI components.
- Create a standard UI according to the rough UI.
- Generate an XML file according to the standard UI.

PROGRESS PRESENTATION 01

- Completed identification of requirements and documentation.
- Sketching, wireframing and prototyping of interfaces.
- Developed a CNN model from the use of python TensorFlow.
- Trained and tested the model by sample data set got from the survey.

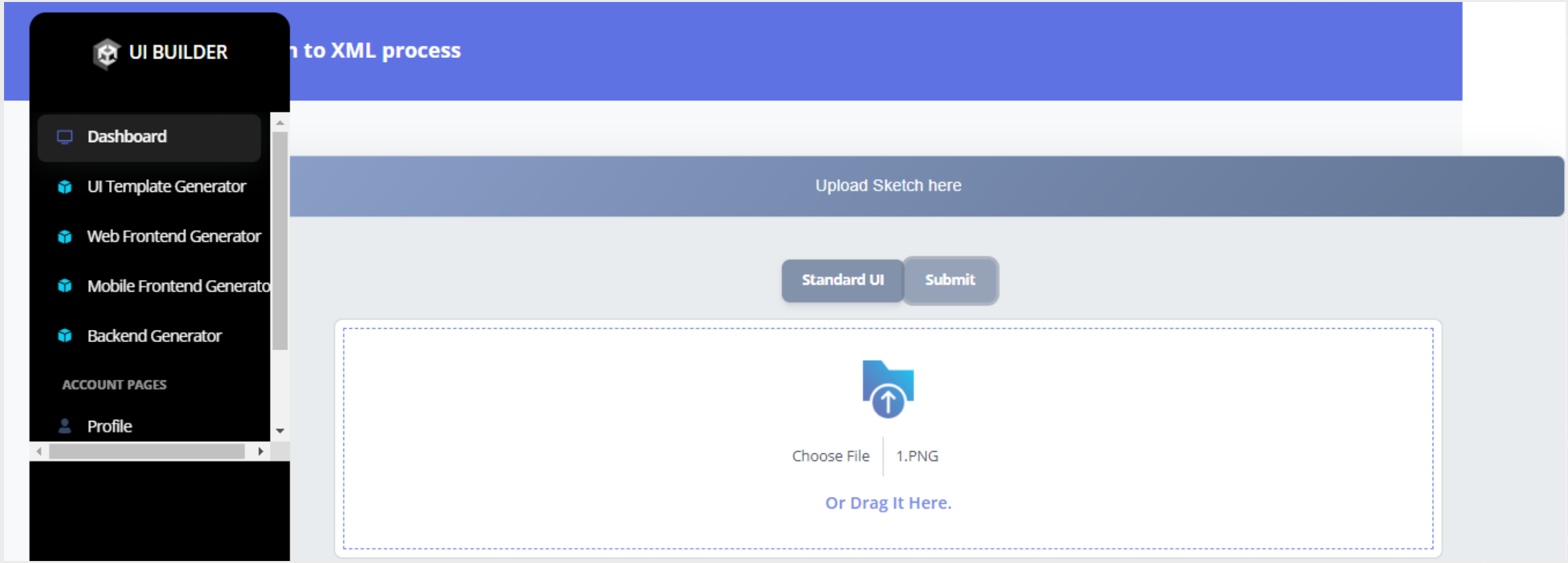


PROGRESS PRESENTATION 02 - DEVELOPMENT

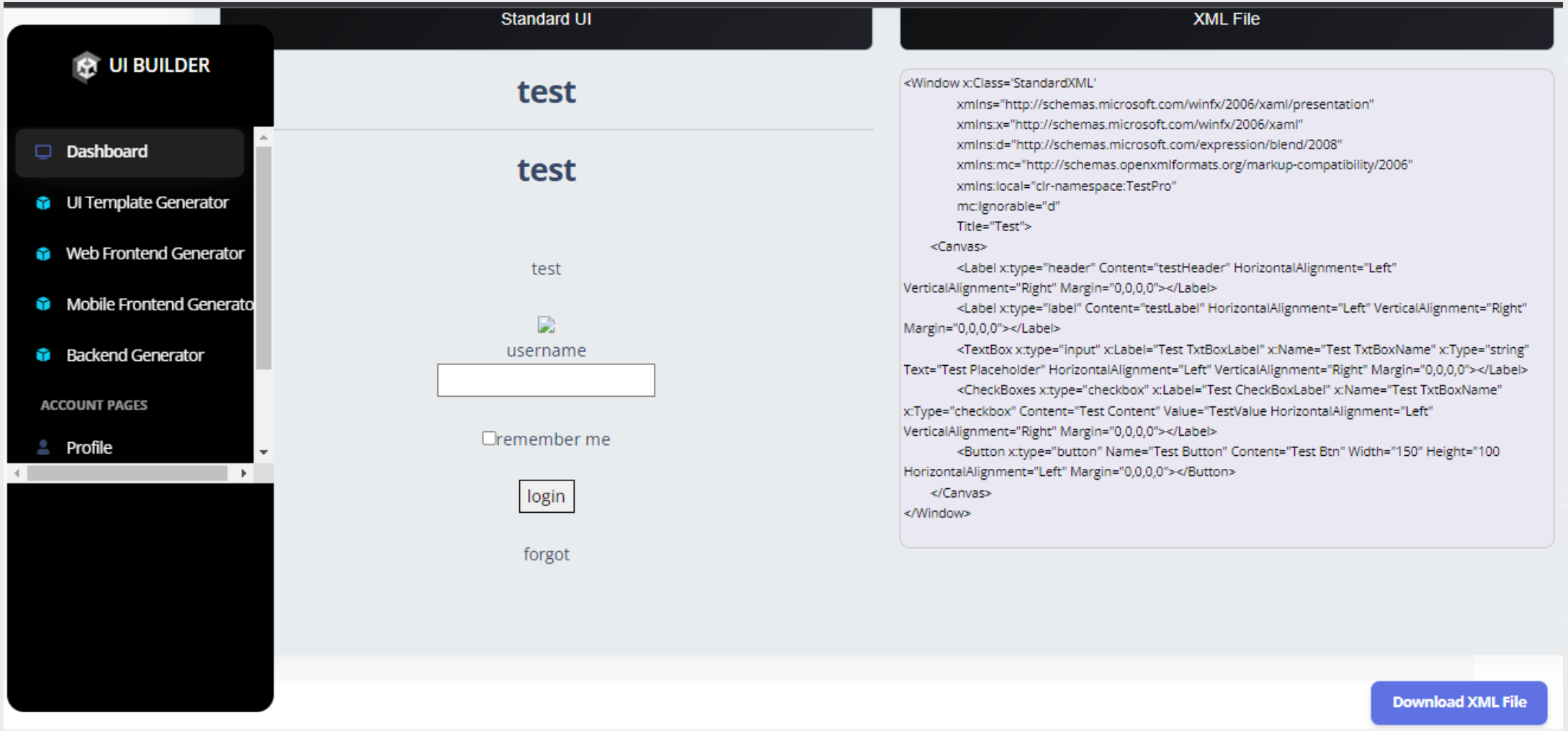
- Completion of training and testing all 23 UI components with the CNN model.
- Completed the react frontend relevant for the component.
- Connect the react frontend with CNN model.
- Tested the predictions from CNN model for different UI sketches.
- Generate Standard UIs.
- Generate XML document for the UIs.



UPLOAD SKETCH UI



DISPLAY STANDARD UI AND XML FILE



REMAINING WORK.



- Perform final user acceptance testing.
- Add optimization techniques requested by end user.
- Consider design suggestions provided by end users
- DOCUMENT : Update the user guide for UI template Generator.
- DOCUMENT : Completion of final report (After supervisor & evaluation panel comments)



IT19158228|SAKTHIVEL.S

BSC (HONS) DEGREE IN INFORMATION TECHNOLOGY

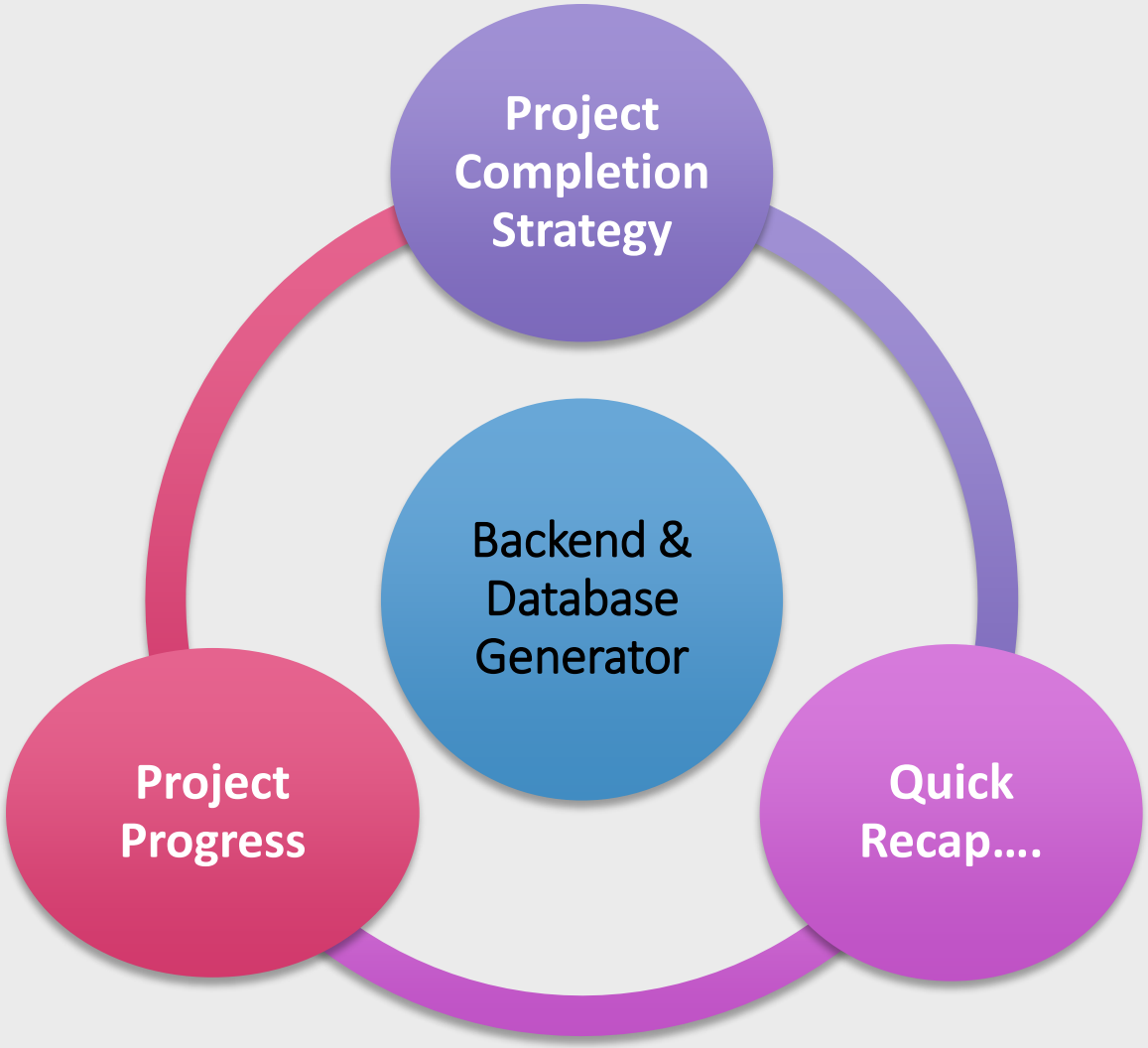
(SPECIALIZATION IN SOFTWARE ENGINEERING)

RESEARCH COMPONENT

BACKEND CODE AND DATABASE GENERATOR



INTRODUCTION



QUICK RECAP.....

Research



- Introduce a backend code generator module for ACG tool.

Research Question



- How to generate backend code for the given UI.

Objectives



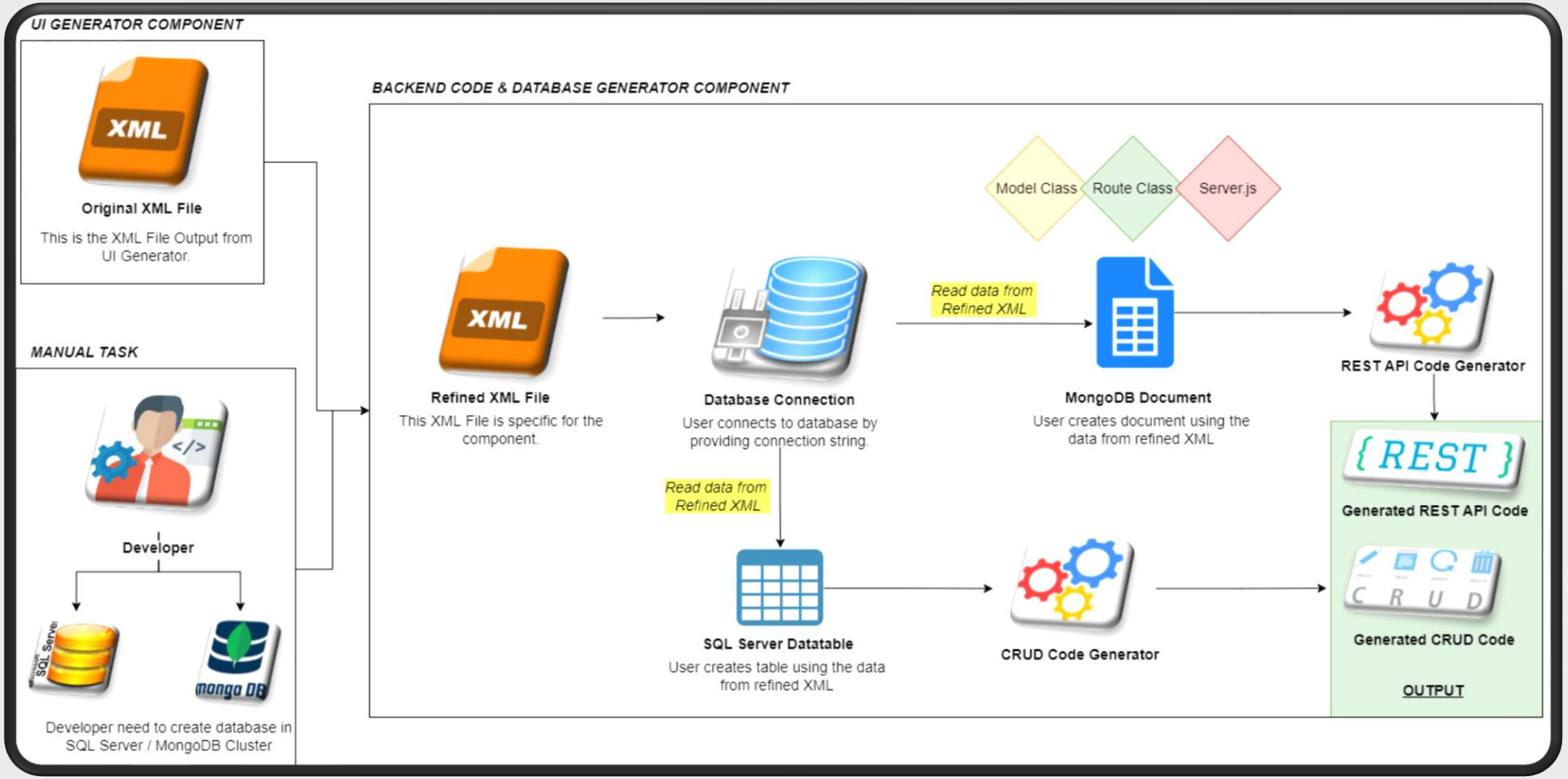
- Generate a fully functional backend code with database connectivity.

Research Gap

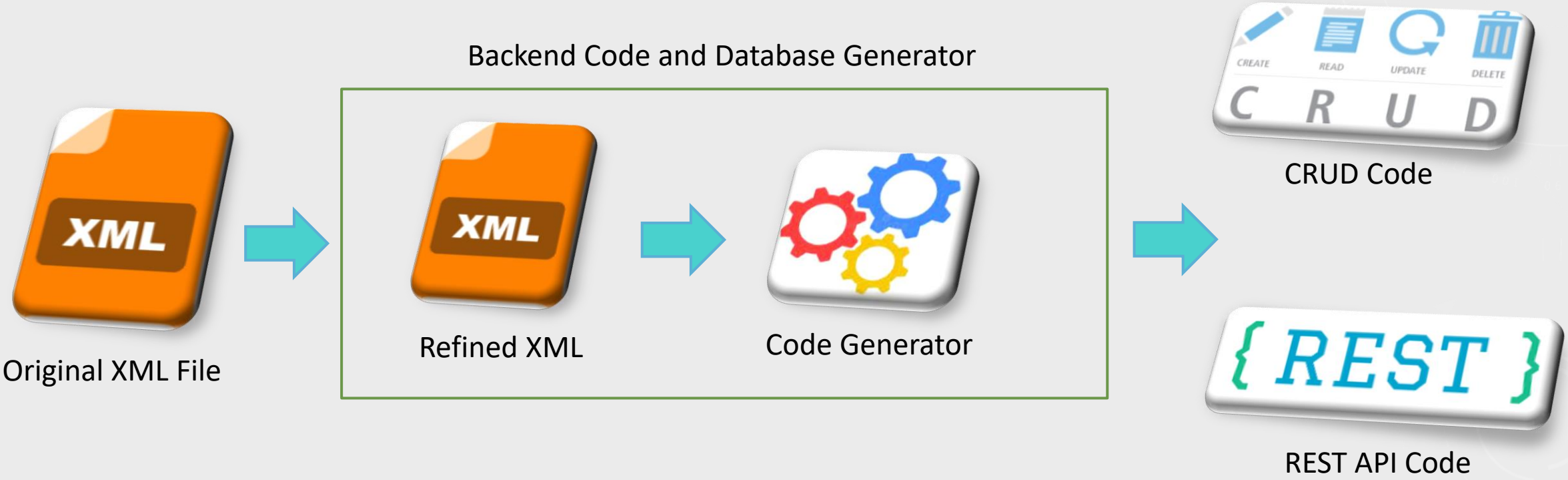


- Existing ACG tools lack backend code generation process.

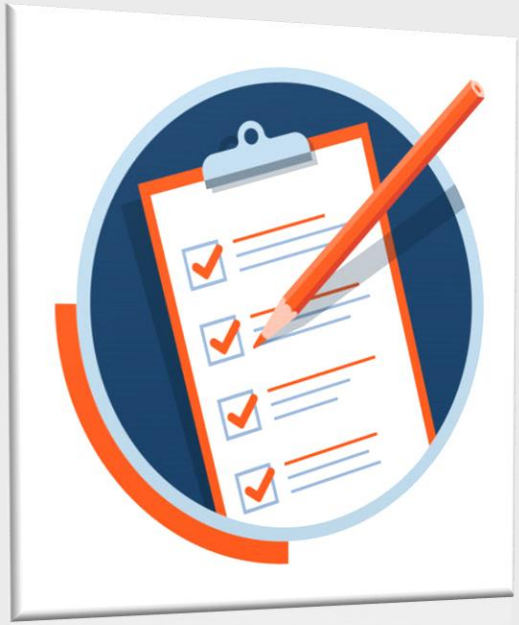
PROCESS


UI GENERATOR COMPONENT
XML
Original XML File
This is the XML File Output from UI Generator.
MANUAL TASK
Developer
Developer need to create database in SQL Server / MongoDB Cluster
BACKEND CODE & DATABASE GENERATOR COMPONENT
Model Class
Route Class
Server.js
XML
Refined XML File
This XML File is specific for the component.
Database Connection
User connects to database by providing connection string.
Read data from Refined XML
MongoDB Document
User creates document using the data from refined XML
REST API Code Generator
{ REST }
Generated REST API Code
Read data from Refined XML
SQL Server Datatable
User creates table using the data from refined XML
CRUD Code Generator
C R U D
Generated CRUD Code
OUTPUT

INPUT OUTPUT



REQUIREMENTS



- Handle original XML and Refined XML file.
- Generate SQL Server or MongoDB connection URL.
- Connect to SQL Server or MongoDB from the module.
- Automatically create table on SQL Server or document on MongoDB.
- Generate CRUD or REST API Code.
- Download the generated code.

PROGRESS PRESENTATION 01

- Completed identification of requirements and documentation.
- Sketching, wireframing and prototyping of interfaces.
- Write the original XML File and Refined XML File. (UI Descriptor)
- Initiated code generation process with UI navigation.
- Completed Database Connection and Table creation process.
- Accomplished CRUD Code generation & automatic comment generation.



PROGRESS PRESENTATION 02 - DEVELOPMENT

- Completion of XML upload and Refined XML file creation.
- Generate MS SQL Server connection URL.
- Revisit Table creation logic with new features (Add columns, keys)
- Completed comment generation.
- Completed CRUD Code generation and REST API Code generation.
- Removed dependencies between the modules.



SAMPLE GENERATED CODE - VIEW IN VS CODE

```
/*
**********************************************************************
*NOTE          :   This is an auto-generated Code
*@Developer    :   (Add Developer Name Here)
*@Purpose      :   Add Customer to database table.
*@Created Date :   5/8/2022 6:48:57 PM
**********************************************************************
*/
public void addCustomer(String custId, String custName, String custAddress, String cutsTel, String dateOfBirth) {
    try{
        SqlConnection con = new SqlConnection(@Data Source=DESKTOP-I53H1O7;Initial Catalog='Test Database';Integrated Security=True);
        SqlCommand cmd;

        String sql = "INSERT INTO Customer VALUES(@custId,@custName,@custAddress,@cutsTel,@dateOfBirth);"
        con.Open();
        cmd = new SqlCommand(sql, con);

        cmd.Parameters.AddWithValue("@custId", custId);
        cmd.Parameters.AddWithValue("@custName", custName);
        cmd.Parameters.AddWithValue("@custAddress", custAddress);
        cmd.Parameters.AddWithValue("@cutsTel", cutsTel);
        cmd.Parameters.AddWithValue("@dateOfBirth", dateOfBirth);

        cmd.ExecuteNonQuery();

        Console.Write('Date Added Successfully!');

    } catch(Exception ex) {
        Console.Write("Exception Occurred!");
    }

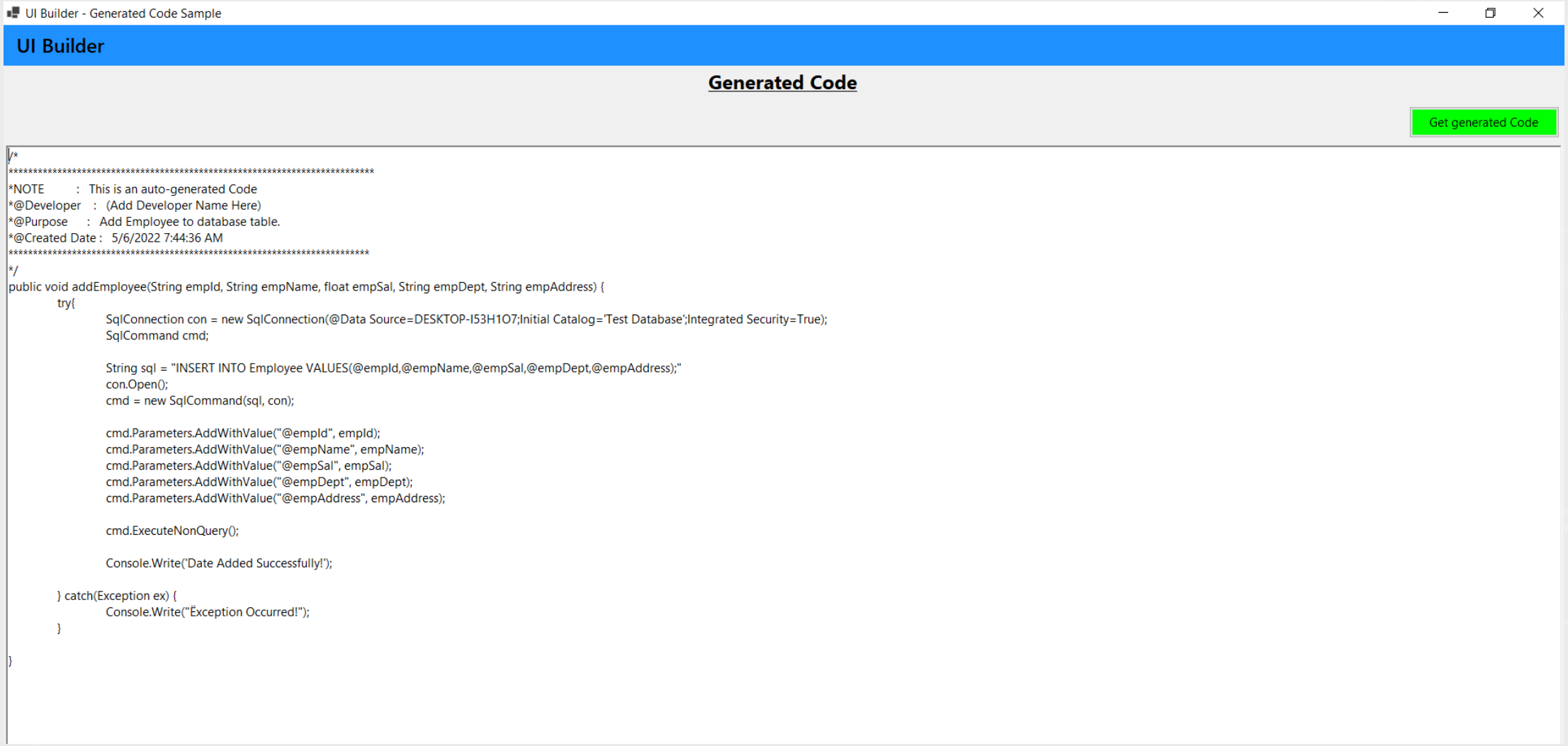
}
```



Observe!

1. **Comments.**
2. **Method Signature.**
3. **Syntax.**
4. **Arrange of Code.**
5. **Error Handling.**

SAMPLE GENERATED CODE – IN APPLICATION




```
/*
**********************************************************************
*NOTE        :  This is an auto-generated Code
*@Developer   :  (Add Developer Name Here)
*@Purpose     :  Add Employee to database table.
*@Created Date :  5/6/2022 7:44:36 AM
**********************************************************************
*/
public void addEmployee(String empId, String empName, float empSal, String empDept, String empAddress) {
        try{
                SqlConnection con = new SqlConnection(@Data Source=DESKTOP-I53H1O7;Initial Catalog='Test Database';Integrated Security=True);
                SqlCommand cmd;

                String sql = "INSERT INTO Employee VALUES(@empId,@empName,@empSal,@empDept,@empAddress);"
                con.Open();
                cmd = new SqlCommand(sql, con);

                cmd.Parameters.AddWithValue("@empId", empId);
                cmd.Parameters.AddWithValue("@empName", empName);
                cmd.Parameters.AddWithValue("@empSal", empSal);
                cmd.Parameters.AddWithValue("@empDept", empDept);
                cmd.Parameters.AddWithValue("@empAddress", empAddress);

                cmd.ExecuteNonQuery();

                Console.Write('Date Added Successfully!');

        } catch(Exception ex) {
                Console.Write("Exception Occurred!");
        }

}
```

REMAINING WORK.

- Perform final testing after completion of all modules.
- Add optimization techniques requested by end user.
- Consider design suggestions provided by end users.
- Focus on non-functional requirements





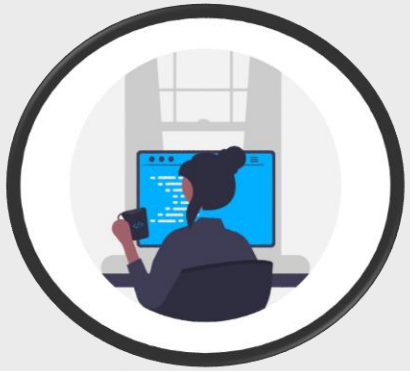
IT19144986|H.M. KASUNI NAVODYA

BSC (HONS) DEGREE IN INFORMATION TECHNOLOGY

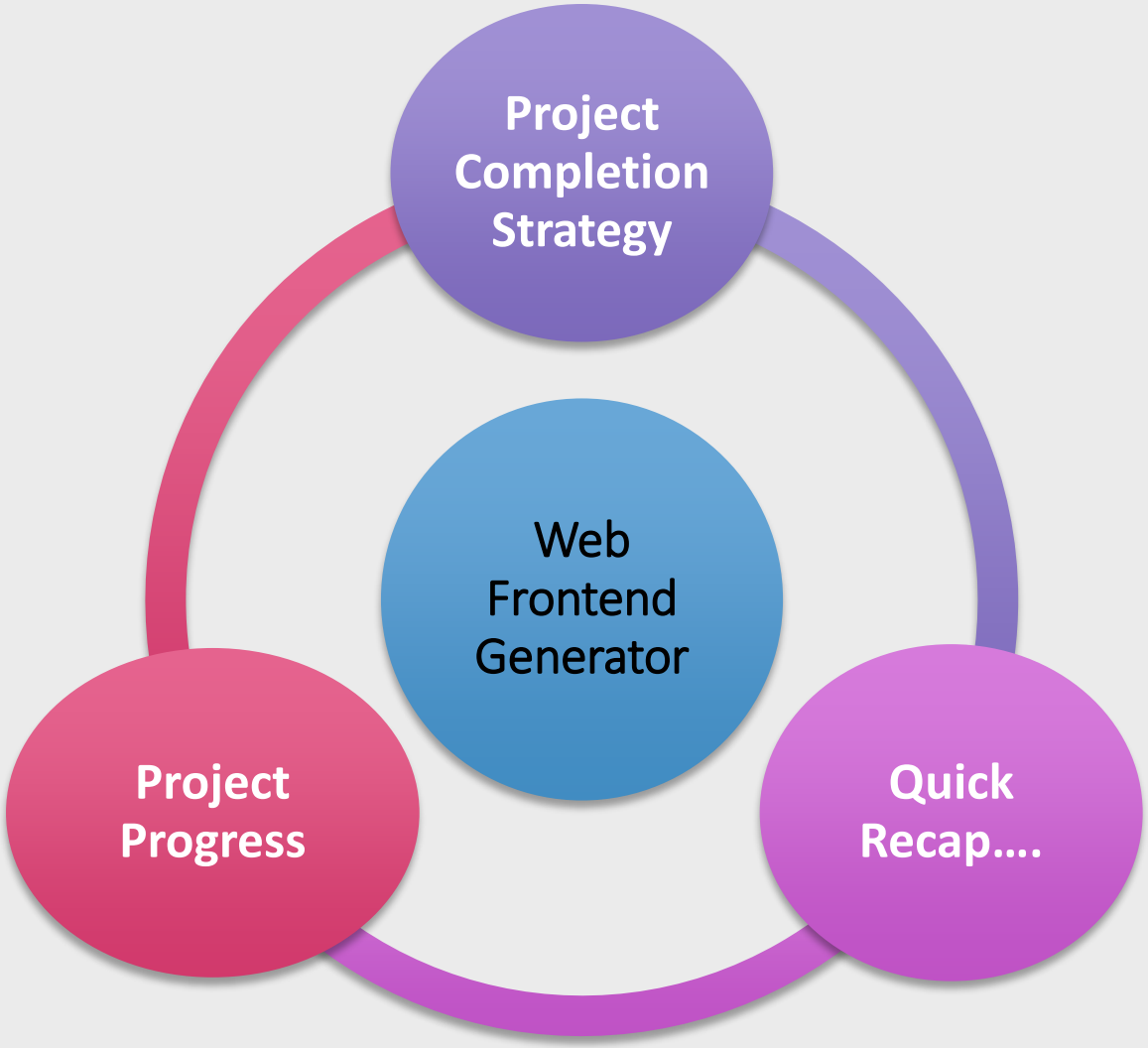
(SPECIALIZATION IN SOFTWARE ENGINEERING)

RESEARCH COMPONENT

FRONT-END CODE GENERATION FOR WEB APPLICATIONS.



INTRODUCTION



QUICK RECAP.....

Research



- Introduce a web frontend generator module for ACG tool.

Research Question



- How to generate web frontend code for the given UI.

Objectives



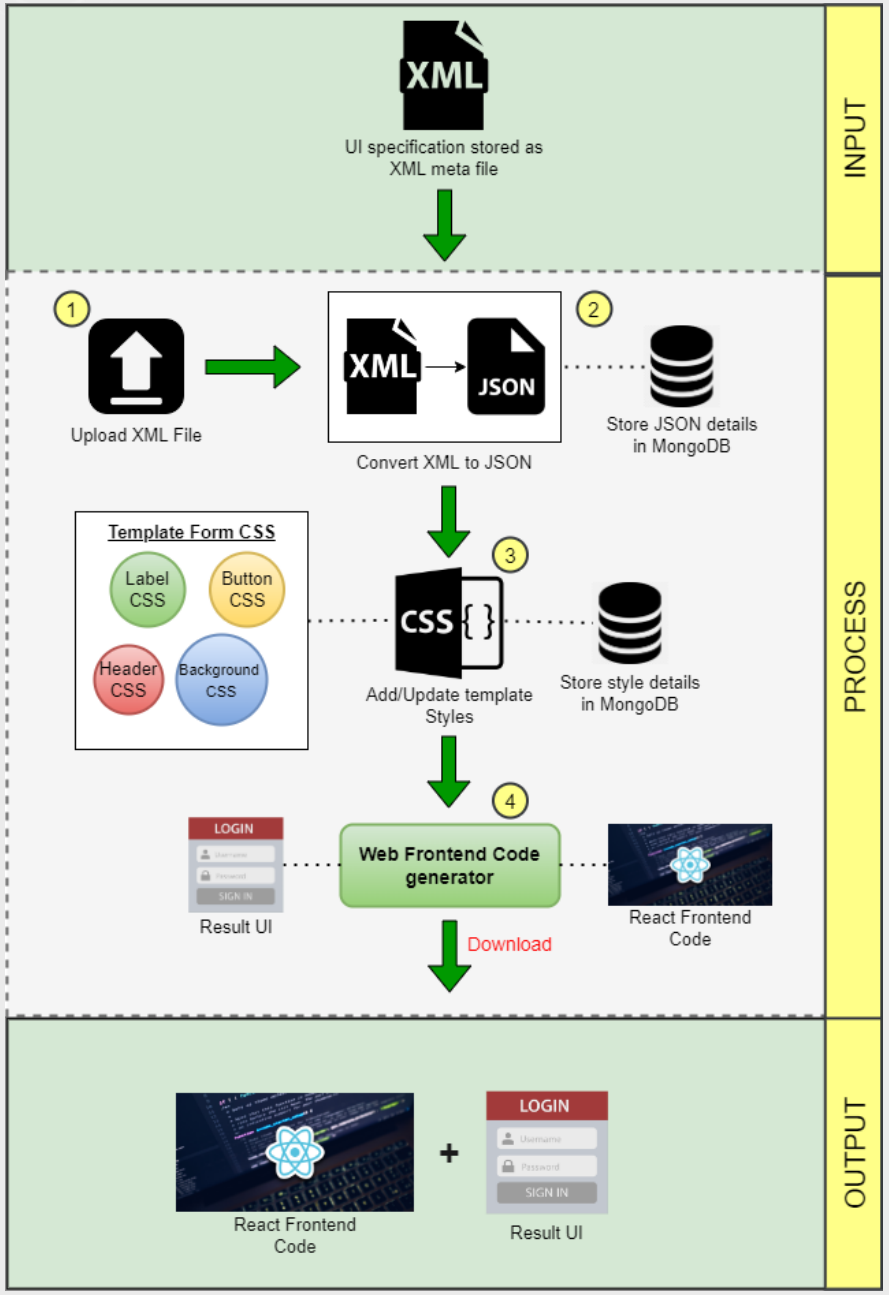
- Generate a simple and accurate automated web frontend code.

Research Gap

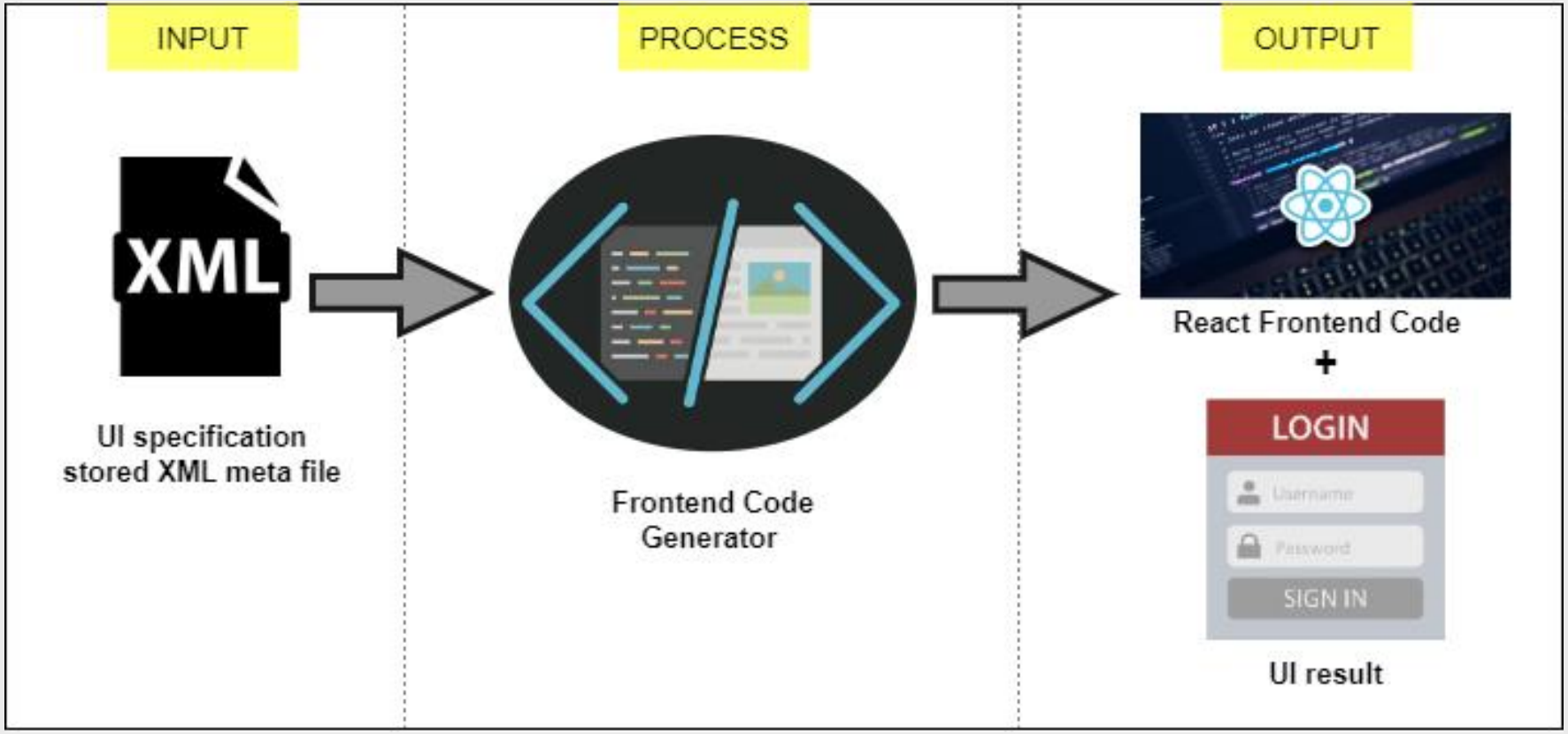


- Absence of flexible & scalable web frontend code generation process.

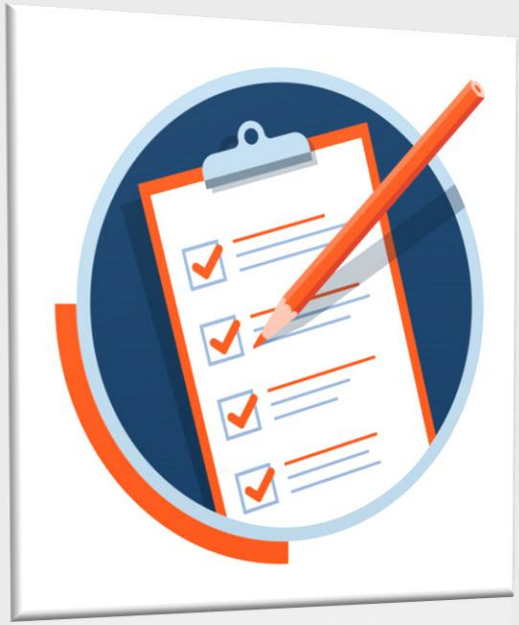
PROCESS


XML
UI specification stored as XML meta file
INPUT
1
Upload XML File
XML
JSON
Convert XML to JSON
2
Store JSON details in MongoDB
Template Form CSS
Label CSS
Button CSS
Header CSS
Background CSS
3
CSS
Add/Update template Styles
Store style details in MongoDB
4
LOGIN
Result UI
Web Frontend Code generator
React Frontend Code
Download
PROCESS
React Frontend Code
+
LOGIN
Result UI
OUTPUT

INPUT OUTPUT



REQUIREMENTS



- Handle original XML and formatted JSON file.
- Manage form styles using pre-defined templates or user selection.
- Generate frontend code for web applications.
- Automatically display the UI result according to the frontend code.
- Download the generated frontend code + UI result

PROGRESS PRESENTATION 01

UI Builder

- Completed identification of requirements and documentation.
- Sketching, wireframing and prototyping of interfaces.
- Completed converting XML to JSON.
- Initiated frontend development with UI navigation.
- Create form styles using pre-defined templates.
- Generate and preview the sample code with UI.



PROGRESS PRESENTATION 02 - DEVELOPMENT

UI Builder

- Completion of XML upload and convert to JSON process.
- Store the JSON details in database.
- Modify form styles using pre-defined templates and user selections.
- Complete the style selection process.
- Complete the web frontend generation process.
- Complete the code and result UI downloading process.



SAMPLE GENERATED CODE – IN APPLICATION

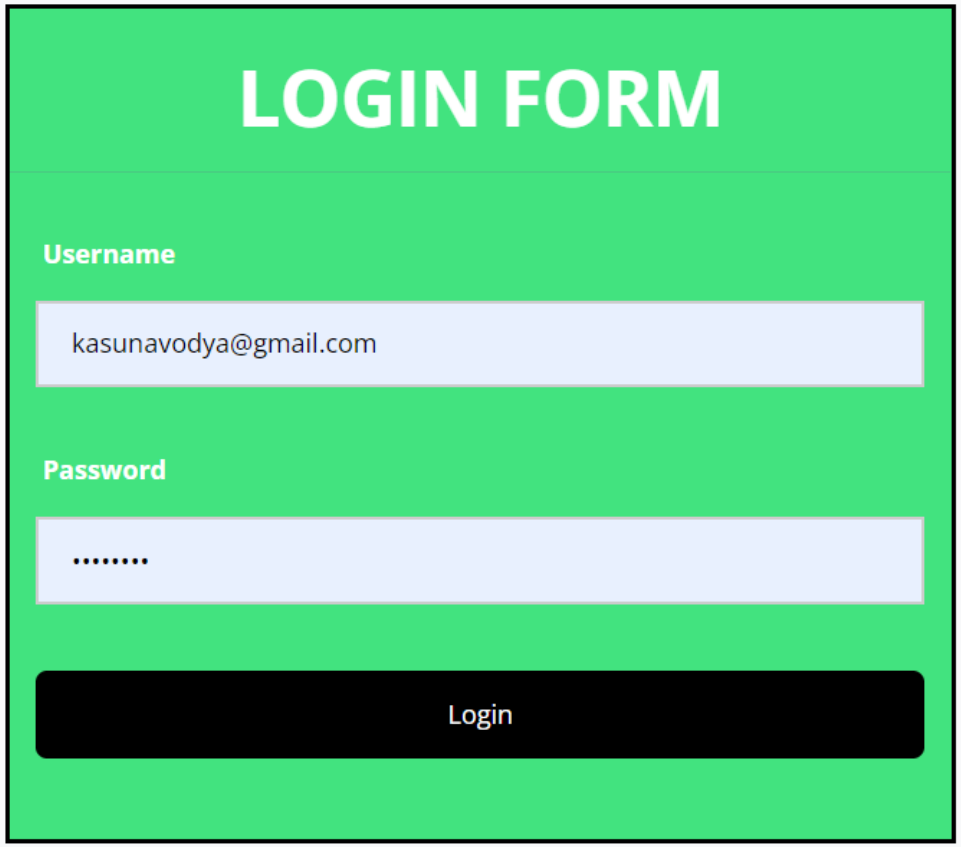
Display React Code

(NOTE: If you want the code screen to be in light mode, click on the code screen.)

Generate Code

```
<div className="LoginWindow">
    <form style={{ border: "2px ridge #000000" , backgroundColor: "#187515", width: "43%" , marginLeft:
"29%" }}>
        <div class="formContainer" style={{textAlign: "center", margin: "24px 0 12px 0"}}>
            <h1 style={{ color: "#f7f7f7" }}>LoginWindow</h1><hr/>
        <div>
        <div class="container" style={{padding: "16px"}}>
            <label style={{ float: "left", paddingTop:"5px", fontSize: "16px" }}>
                <b style={{ color: "#ffffff"}}>Username</b>
            </label>
            <input
                type="text"
                placeholder="Enter Username"
                name="Username"
                required
                style={{ padding: "12px 20px", margin: "8px 0", width: "100%", border: "2px groove
#939599"}}
            />
            <br /><br />
            <label style={{ float: "left", paddingTop:"5px", fontSize: "16px" }}>
                <b style={{ color: "#ffffff"}}>Password</b>
            </label>
            <input
                type="text"
                placeholder="Enter Password"
                name="Password"
                required
                style={{ padding: "12px 20px", margin: "8px 0", width: "100%", border: "2px groove
#939599"}}
```

Display Result

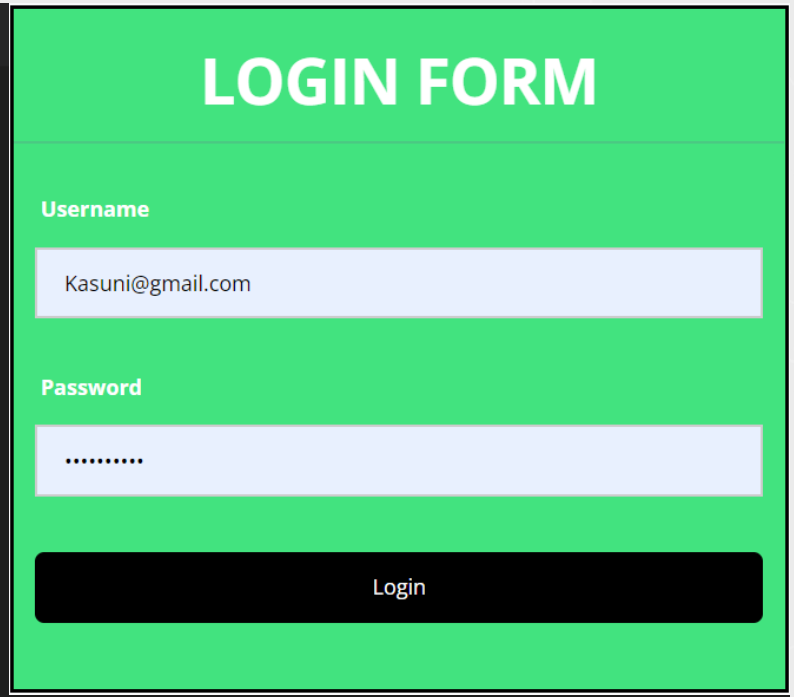


SAMPLE GENERATED CODE - VIEW IN VS CODE

Web_Frontend_Code.js

C: > Users > kasun > Downloads > Web_Frontend_Code.js > ...

```
<div className="LoginForm">
    <form style={{ border: '3px solid #000000', backgroundColor: '#42e37f', width: '43%', marginLeft: '29%' }}>
        <div class="formContainer" style={{ textAlign: 'center', margin: '24px 0 12px 0' }}>
            <h1 style={{ color: 'white' }}>LOGIN FORM</h1><hr/>
        </div>
        <div class="container" style={{ padding: '16px' }}>
            <label for="uname" style={{ float: 'left', paddingTop: '5px', fontSize: '16px' }}>
                <b style={{ color: 'white' }}>Username</b>
            </label>
            <input
                type="text"
                placeholder="Enter Username"
                name="uname"
                required
                style={{ padding: '12px 20px', margin: '8px 0', border: '2px solid #ccc', width: '100%' }}
            />
            <br /><br />
            <label for="psw" style={{ float: 'left', paddingTop: '5px', fontSize: '16px' }}>
                <b style={{ color: 'white' }}>Password</b>
            </label>
            <input
                type="password"
                placeholder="Enter Password"
                name="psw"
                required
                style={{ padding: '12px 20px', margin: '8px 0', border: '2px solid #ccc', width: '100%' }}
            />
            <br /><br />
            <button type="submit" style={{ backgroundColor: '#000000', color: 'white', padding: '14px 20px', margin: '8px 0', border: 'none', borderRadius: '7px', width: '100%' }}>Login</button>
            <br /><br /><br />
        </div>
    </form>
</div>
```



REMAINING WORK.



- Perform system testing after completion of all modules.
- Add optimization techniques requested by end user.
- Consider UI/UX suggestions provided by end users.
- DOCUMENT : Update the user guide for web frontend code generator.
- DOCUMENT : Completion of final report. (After supervisor & evaluation panel comments)



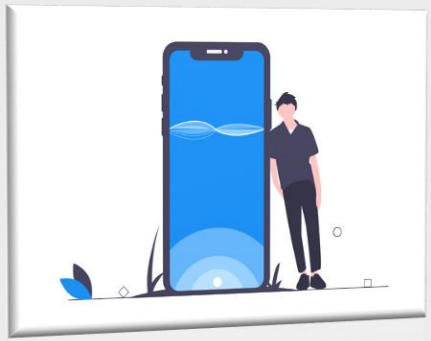
IT19150826|D.P. KAVINDI GIMSHANI

BSC (HONS) DEGREE IN INFORMATION TECHNOLOGY

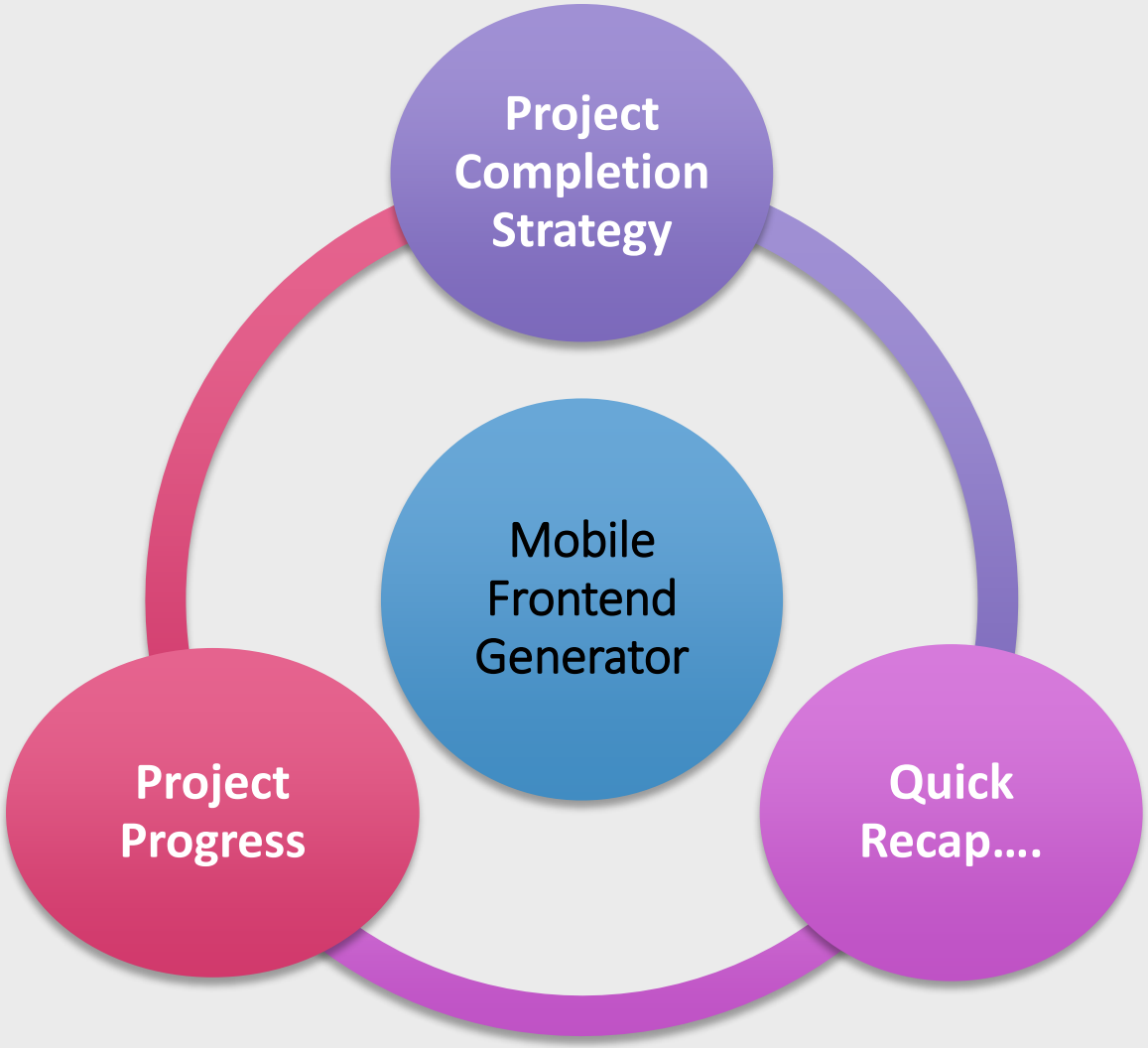
(SPECIALIZATION IN SOFTWARE ENGINEERING)

RESEARCH COMPONENT

FRONT-END CODE GENERATION FOR MOBILE APPLICATIONS USING CODE MAPPING



INTRODUCTION



QUICK RECAP.....

Research



- Introduce a mobile frontend generator module for ACG tool.

Research Question



- How to generate mobile frontend code for the given UI.

Objectives



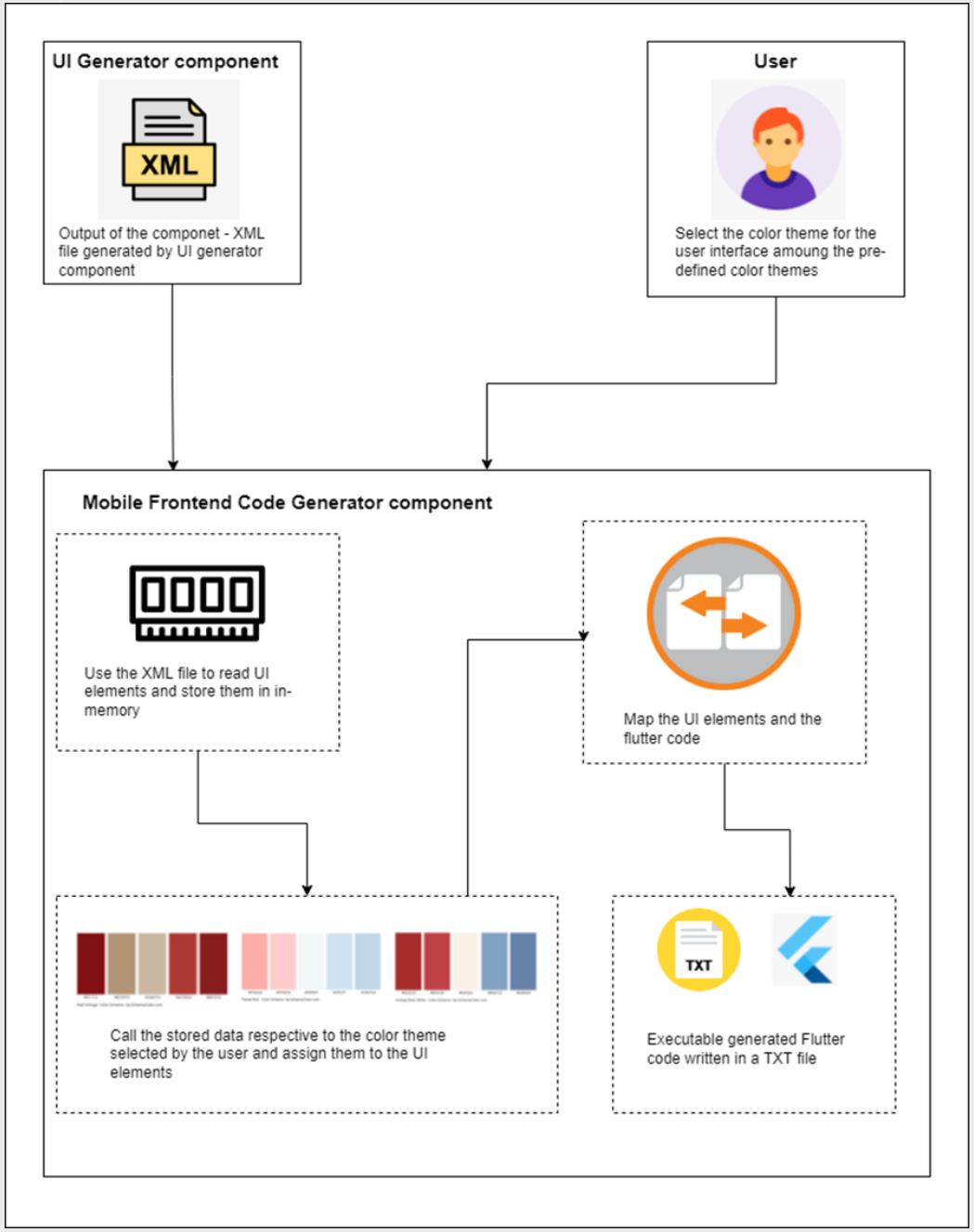
- Generate a simple and accurate automated mobile frontend code.

Research Gap

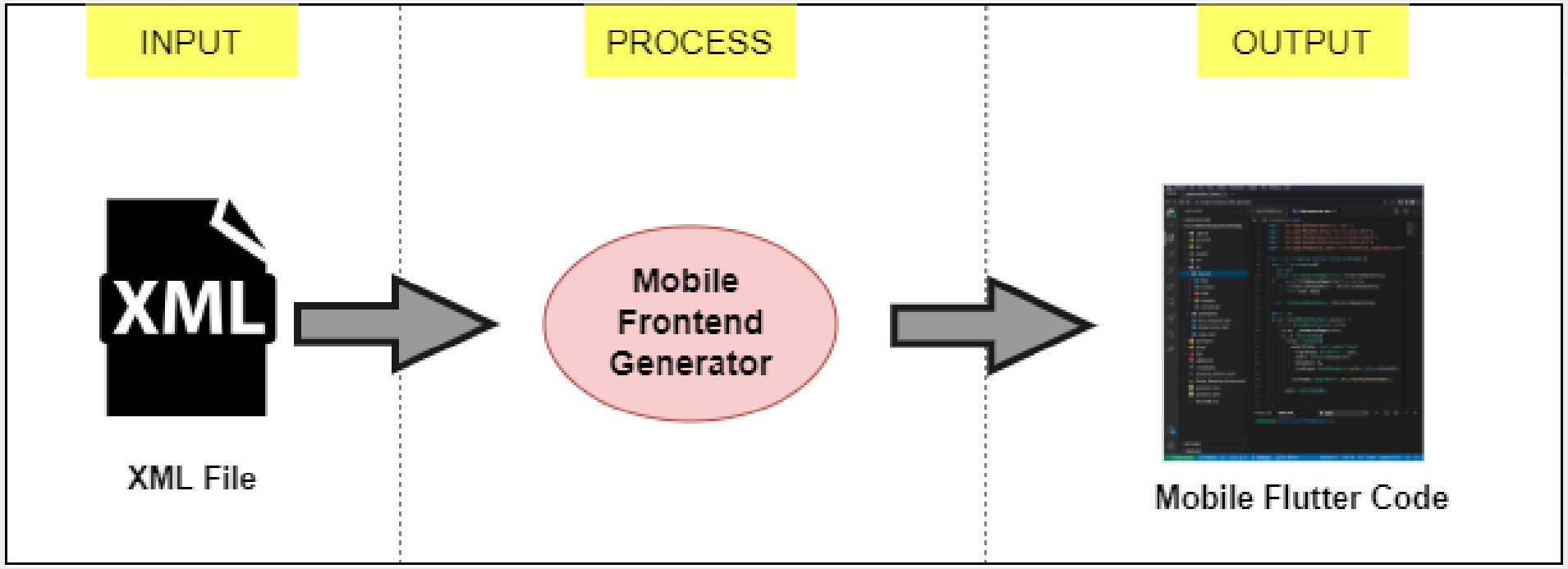


- Absence of flexible & scalable mobile frontend code generation process.

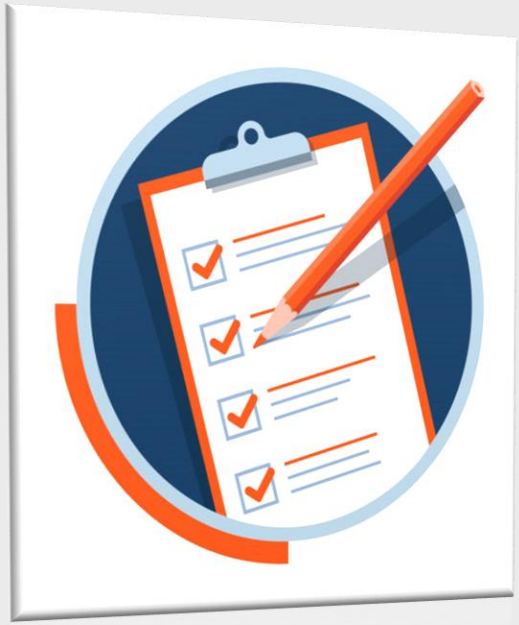
PROCESS


UI Generator component
XML
Output of the componet - XML file generated by UI generator component
User
Select the color theme for the user interface amoung the pre-defined color themes
Mobile Frontend Code Generator component
Use the XML file to read UI elements and store them in in-memory
Map the UI elements and the flutter code
Call the stored data respective to the color theme selected by the user and assign them to the UI elements
TXT
Executable generated Flutter code written in a TXT file

INPUT OUTPUT



REQUIREMENTS



- Read the XML file
- Allow user to choose the color theme
- **Generate frontend code for** mobile applications
- Display the generates code in the system
- Allow user to choose a location and store the generated code

PROGRESS PRESENTATION 01

UI Builder

- Completed identification of requirements and documentation.
- Sketching, wireframing and prototyping of interfaces.
- Completed reading XML file.
- Initiated frontend development with UI navigation.
- Create color themes using pre-defined templates.
- Generate and preview the sample code with UI.



PROGRESS PRESENTATION 02 - DEVELOPMENT

- Completed uploading and reading XML file.
- Completed the color theme selection process.
- Modified UI using pre-defined templates and user selections.
- Completed the mobile frontend generation process.
- Completed the generated code display process.
- Completed the generated code storing in user's local machine process.



SAMPLE GENERATED CODE - VIEW IN ANDROID STUDIO

main.dart

```dart
import 'package:flutter/material.dart';

void main() {
  runApp( MyApp());
}

class MyApp extends StatelessWidget {
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        backgroundColor: Color(0xFFffb997),
        appBar: AppBar(
        title: const Text('Welcome to My App'),
        ),  // AppBar
        body: MyCustomForm(),
      ),  // Scaffold
    );  // MaterialApp
  }}

// Create a Form widget.
class MyCustomForm extends StatefulWidget{
  MyCustomFormState createState() {
    return MyCustomFormState();
  }
}

class MyCustomFormState extends State<MyCustomForm> {
```

main.dart

```dart
class MyCustomFormState extends State<MyCustomForm> {
  //a global key that uniquely identifies the Form widget and allows validation of the form
  final _formKey = GlobalKey<FormState>();
  Widget build(BuildContext context) {
    return Form(
      key: _formKey,
      child: Center( child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
        const Text( 'Login Form',
            style: TextStyle(
                color: Color(0xFF621940),
                fontSize: 30,
                fontWeight: FontWeight.bold)  // TextStyle
        ),  // Text
        const Divider(
            color: Color(0xFF0b032d)
        ),  // Divider
        const Text( 'Username',
            style: TextStyle(
              color: Color(0xFF000000),
              fontSize: 20,
            )  // TextStyle
        ),  // Text
        TextFormField(
          decoration: const InputDecoration(
```

SAMPLE GENERATED CODE – IN APPLICATION




```
import 'package:flutter/material.dart';

void main() {
        runApp( MyApp());
}

class MyApp extends StatelessWidget {
        Widget build(BuildContext context) {
        return MaterialApp(
                home: Scaffold(
                                backgroundColor: Color(0xFFc1e3ed),
                        appBar: AppBar(
                                title: const Text('Welcome to My App'),
                        ),
                        body: MyCustomForm(),
                        ),
        );
 }}

// Create a Form widget.
class MyCustomForm extends StatefulWidget{
        MyCustomFormState createState() {
                return MyCustomFormState();
        }
}

class MyCustomFormState extends State<MyCustomForm> {
        //a global key that uniquely identifies the Form widget and allows validation of the form
        final _formKey = GlobalKey<FormState>();
        Widget build(BuildContext context) {
         return Form(
                key: _formKey,
                child: Center( child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
```

PROGRESS PRESENTATION 02 - GROUP

- ✓Completed code commenting and refactoring.
- ✓Manage GitLab's repository branch.
- ✓Checked for interoperability.
- ✓Redesigned UI considering user experiences.
- ✓Change in workflows and critical logics.
- ✓Added new features for completeness of the module.



REMAINING WORK - GROUP



- Perform system testing after completion of all modules.
- Re-engineer flows with end user requests.
- Consider UI/UX suggestions provided by end users.
- Try automatic frontend and backend code binding.
- Completion of Project documentation (Final report, Project status document)

COMMERCIALIZATION.

Silver — Standard

- Free.
- UI Generator and Backend code generator available.
- Maximum 4 projects.
- Code regeneration is disabled.

Gold — Community

- 23.50$ per year (Subscription based)
- UI Generator, Frontend code generator and backend code generator available,
- Maximum 15 projects within subscription period.
- Code regeneration and code refactor.

Platinum — Enterprise

- 50$ per year + 24x7 Customer support.
- All features are readily available.
- Unlimited projects.
- Offline support.



REFERENCES

[1]. Beltramelli, Tony. "pix2code: Generating code from a graphical user interface screenshot." Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems. 2018.

[2]. d'Amorim, Marcelo, Rui Abreu, and Carlos Mello. "Visual sketching: from image sketches to code." Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results. 2020.

[3]. Uyanik, Burak, and Veysel Harun Şahin. "A template-based code generator for web applications." Turkish Journal of Electrical Engineering & Computer Sciences 28.3 (2020): 1747-1762.

[4]. Aşıroğlu, Batuhan, et al. "Automatic HTML code generation from mock-up images using machine learning techniques." 2019 Scientific Meeting on Electrical-Electronics & Biomedical Engineering and Computer Science (EBBT). Ieee, 2019.

[5]. Beltramelli, Tony. (2017). pix2code: Generating Code from a Graphical User Interface Screenshot.

[6]. Deng, Youqiang & Fu, Cai & Li, Yang. (2019). CodeeGAN: Code Generation via Adversarial Training. 10.1007/978-981-15-1304-6_2.

[7]. B. Aşıroğlu et al., "Automatic HTML Code Generation from Mock-Up Images Using Machine Learning Techniques," 2019 Scientific Meeting on Electrical-Electronics & Biomedical Engineering and Computer Science (EBBT), 2019, pp. 1-4, doi: 10.1109/EBBT.2019.8741736.

[8]. M. d'Amorim, R. Abreu and C. Mello, "Visual Sketching: From Image Sketches to Code," 2020 IEEE/ACM 42nd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER), 2020, pp. 101-104.

[9]. B. Vogel-Heuser, D. Witsch and U. Katzke, "Automatic code generation from a UML model to IEC 61131-3 and system configuration tools," 2005 International Conference on Control and Automation, 2005, pp. 1034-1039 Vol. 2, doi: 10.1109/ICCA.2005.1528274.

[10]. Jordi Cabot, “AI-based tools to transform interface design mockups into ready-to-use UI code.” https://modeling-languages.com/ai-tools-transform-design-mockups-into-ui-code/ [Accessed: 20 – Jan - 2022]

[11]. Noah Gundotra, "Code2Pix — Deep Learning Compiler for Graphical User Interfaces." https://towardsdatascience.com/code2pix-deep-learning-compiler-for-graphical-user-interfaces-1256c346950b [Accessed: 20 – Jan - 2022]

[12]. Vanita Jain, Piyush Agrawal, Subham Banga, Rishabh Kapoor, Shashwat Gulyani, "Sketch2Code: Transformation of Sketches to UI in Real-time Using Deep Neural Network." https://arxiv.org/abs/1910.08930 [Accessed: 20 – Jan - 2022]

[13]. M. Usman, A. Nadeem and T. -h. Kim, "UJECTOR: A Tool for Executable Code Generation from UML Models," 2008 Advanced Software Engineering and Its Applications, 2008, pp. 165-170, doi: 10.1109/ASEA.2008.39.

[14]. S. D. Rathod, "Automatic code generation with business logic by capturing attributes from user interface via XML," 2016 International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT), 2016, pp. 1480-1484, doi: 10.1109/ICEEOT.2016.7754929.

[15]. IFML: The Interaction Flow Modeling Language: https://www.ifml.org/

[16]. Behrens, Heiko. "MDSD for the iPhone: developing a domain-specific language and IDE tooling to produce real world applications for mobile devices." *Proceedings of the ACM international conference companion on Object oriented programming systems languages and applications companion*. 2010.

[17]. Parada, Abilio, Milena Marques, and Lisane B. de Brisolara. "Automating mobile application development: UML-based code generation for Android and Windows Phone." *Revista de Informática Teórica e Aplicada* 22.2 (2015): 31-50.

THANK YOU..



Q & A.

DEMONSTRATION

